

Конечно! Давайте сделаем код максимально подробным и понятным, добавив комментарии к каждому шагу. Это поможет вашим студентам лучше разобраться в материале.

Лекция по JavaScript (Часть 6)

Продолжительность: 3 часа

Предыдущие темы:

1. Введение в JavaScript: синтаксис, переменные, типы данных, операторы.
 2. Управляющие конструкции: условия, циклы, функции.
 3. Массивы, объекты, работа с DOM, асинхронность.
 4. Модули, замыкания, классы и ООП.
 5. Работа с API, обработка ошибок, итераторы, генераторы, прокси.
-

План лекции:

1. Работа с файлами и формами (1 час)

- Чтение файлов с использованием `<input type="file">`.
- Валидация форм.
- Отправка данных формы на сервер.
- Практические примеры.

2. Веб-сокеты и реальное время (1 час)

- Что такое веб-сокеты?
- Создание простого чата с использованием веб-сокетов.
- Практические примеры.

3. Оптимизация производительности в JavaScript (1 час)

- Оптимизация загрузки и выполнения кода.
 - Ленивая загрузка (lazy loading).
 - Оптимизация работы с DOM.
 - Практические примеры.
-

Подробное содержание лекции с примерами:

1. Работа с файлами и формами (1 час)

Теория:

Файлы:

- JavaScript позволяет работать с файлами, загруженными пользователем через `<input type="file">`.
- Можно читать содержимое файлов (текст, изображения и т.д.).

Формы:

- Формы используются для сбора данных от пользователя.
- JavaScript позволяет валидировать данные перед отправкой на сервер.

Пример 1: Чтение файлов

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>File Reader Example</title>
</head>
<body>
  <h1>Upload a File</h1>
  <!-- Поле для выбора файла -->
  <input type="file" id="file-input">
  <!-- Элемент для отображения содержимого файла -->
  <pre id="file-content"></pre>

  <script>
    // Получаем элементы DOM
    const fileInput = document.getElementById('file-input');
    const fileContent = document.getElementById('file-content');

    // Добавляем обработчик события изменения файла
    fileInput.addEventListener('change', (event) => {
      // Получаем выбранный файл
      const file = event.target.files[0];
```

```

        // Создаём объект FileReader для чтения файла
        const reader = new FileReader();

        // Обработчик события, который срабатывает, когда файл прочитан
        reader.onload = (e) => {
            // Выводим содержимое файла в элемент <pre>
            fileContent.textContent = e.target.result;
        };

        // Читаем файл как текст
        reader.readAsText(file);
    });
</script>
</body>
</html>

```

Комментарии:

- Пользователь выбирает файл через `<input type="file">`.
- Используем `FileReader` для чтения содержимого файла.
- Содержимое файла выводится в элемент `<pre>`.

Пример 2: Валидация формы

```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Form Validation Example</title>
    <style>
        .error { color: red; }
    </style>
</head>
<body>
    <h1>Registration Form</h1>
    <!-- Форма для регистрации -->
    <form id="registration-form">
        <label for="username">Username:</label>
        <input type="text" id="username" name="username" required>
        <span id="username-error" class="error"></span><br>

        <label for="email">Email:</label>
        <input type="email" id="email" name="email" required>
    </form>

```

```

<span id="email-error" class="error"></span><br>

<label for="password">Password:</label>
<input type="password" id="password" name="password" required>
<span id="password-error" class="error"></span><br>

<button type="submit">Register</button>
</form>

<script>
  // Получаем форму
  const form = document.getElementById('registration-form');

  // Добавляем обработчик события отправки формы
  form.addEventListener('submit', (event) => {
    event.preventDefault(); // Отменяем стандартную отправку формы

    // Получаем значения полей формы
    const username = form.username.value.trim();
    const email = form.email.value.trim();
    const password = form.password.value.trim();

    let isValid = true; // Флаг для проверки валидности формы

    // Валидация имени пользователя
    if (username.length < 3) {
      document.getElementById('username-error').textContent =
'Username must be at least 3 characters long.';
      isValid = false;
    } else {
      document.getElementById('username-error').textContent = '';
    }

    // Валидация email
    if (!email.includes('@')) {
      document.getElementById('email-error').textContent = 'Please
enter a valid email address.';
      isValid = false;
    } else {
      document.getElementById('email-error').textContent = '';
    }

    // Валидация пароля
    if (password.length < 6) {
      document.getElementById('password-error').textContent =
'Password must be at least 6 characters long.';

```

```
        isValid = false;
    } else {
        document.getElementById('password-error').textContent = '';
    }

    // Если форма валидна, отправляем данные
    if (isValid) {
        alert('Form submitted successfully!');
        // Здесь можно отправить данные на сервер
    }
    });
</script>
</body>
</html>
```

Комментарии:

- Форма содержит поля для имени пользователя, email и пароля.
 - При отправке формы выполняется валидация данных.
 - Если данные невалидны, выводятся сообщения об ошибках.
 - Если данные валидны, форма считается успешно отправленной.
-

2. Веб-сокеты и реальное время (1 час)

Теория:

Веб-сокеты:

- Это протокол для двусторонней связи между клиентом и сервером в реальном времени.
- Используется для чатов, уведомлений, онлайн-игр и т.д.

Пример: Простой чат с использованием веб-сокетов

Серверная часть (Node.js):

```
const WebSocket = require('ws');
const wss = new WebSocket.Server({ port: 8080 });

// Обработчик подключения нового клиента
wss.on('connection', (ws) => {
    console.log('New client connected');
```

```

// Обработчик получения сообщения от клиента
ws.on('message', (message) => {
    console.log(`Received: ${message}`);
    // Отправляем сообщение всем подключённым клиентам
    wss.clients.forEach(client => {
        if (client.readyState === WebSocket.OPEN) {
            client.send(message);
        }
    });
});

// Обработчик отключения клиента
ws.on('close', () => {
    console.log('Client disconnected');
});
});

```

Клиентская часть (HTML + JavaScript):

```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>WebSocket Chat</title>
</head>
<body>
    <h1>WebSocket Chat</h1>
    <div id="chat">
        <ul id="messages"></ul>
    </div>
    <input type="text" id="message-input" placeholder="Type a message...">
    <button id="send-button">Send</button>

    <script>
        // Создаём соединение с веб-сокет сервером
        const ws = new WebSocket('ws://localhost:8080');

        // Получаем элементы DOM
        const messages = document.getElementById('messages');
        const messageInput = document.getElementById('message-input');
        const sendButton = document.getElementById('send-button');

        // Обработчик получения сообщения от сервера
    </script>

```

```
ws.onmessage = (event) => {
    const message = document.createElement('li');
    message.textContent = event.data;
    messages.appendChild(message);
};

// Обработчик нажатия на кнопку отправки
sendButton.addEventListener('click', () => {
    const message = messageInput.value;
    ws.send(message); // Отправляем сообщение на сервер
    messageInput.value = ''; // Очищаем поле ввода
});
</script>
</body>
</html>
```

Комментарии:

- Сервер на Node.js использует библиотеку `ws` для создания веб-сокета-сервера.
 - Клиент подключается к серверу и отправляет сообщения, которые рассылаются всем подключённым клиентам.
 - Сообщения отображаются в виде списка на странице.
-

3. Оптимизация производительности в JavaScript (1 час)

Теория:

Оптимизация загрузки и выполнения кода:

- Используйте асинхронную загрузку скриптов (`async` , `defer`).
- Минифицируйте и сжимайте JavaScript-файлы.

Ленивая загрузка (lazy loading):

- Загружайте ресурсы (изображения, скрипты) только тогда, когда они нужны.

Оптимизация работы с DOM:

- Минимизируйте количество операций с DOM.
- Используйте фрагменты (`DocumentFragment`) для группового добавления элементов.

Пример 1: Ленивая загрузка изображений

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Lazy Loading Images</title>
</head>
<body>
  <h1>Lazy Loading Images</h1>
  
  
  

  <script>
    // Обработчик события, который срабатывает, когда DOM загружен
    document.addEventListener('DOMContentLoaded', () => {
      // Получаем все изображения с классом lazy
      const lazyImages = document.querySelectorAll('.lazy');

      // Создаём IntersectionObserver для отслеживания видимости
      элементов
      const observer = new IntersectionObserver((entries) => {
        entries.forEach(entry => {
          if (entry.isIntersecting) {
            const img = entry.target;
            img.src = img.dataset.src; // Загружаем изображение
            observer.unobserve(img); // Прекращаем наблюдение за
            изображением
          }
        });
      });

      // Начинаем наблюдение за каждым изображением
      lazyImages.forEach(img => observer.observe(img));
    });
  </script>
</body>
</html>

```

Комментарии:

- Изображения загружаются только тогда, когда они появляются в области видимости.
- Используется `IntersectionObserver` для отслеживания видимости элементов.

Пример 2: Оптимизация работы с DOM


```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>DOM Optimization</title>
</head>
<body>
  <h1>DOM Optimization</h1>
  <ul id="list"></ul>

  <script>
    // Получаем элемент списка
    const list = document.getElementById('list');

    // Создаём фрагмент документа для группового добавления элементов
    const fragment = document.createDocumentFragment();

    // Создаём 1000 элементов списка
    for (let i = 1; i <= 1000; i++) {
      const li = document.createElement('li');
      li.textContent = `Item ${i}`;
      fragment.appendChild(li); // Добавляем элемент в фрагмент
    }

    // Добавляем фрагмент в DOM
    list.appendChild(fragment);
  </script>
</body>
</html>
```

Комментарии:

- Используем `DocumentFragment` для группового добавления элементов в DOM.
- Это уменьшает количество операций с DOM и повышает производительность.

Итог:

- **Файлы и формы** — важная часть взаимодействия с пользователем.
- **Веб-сокеты** позволяют создавать приложения в реальном времени.
- **Оптимизация производительности** делает ваши приложения быстрее и эффективнее.

Домашнее задание:

1. Создайте форму для загрузки изображений и отображайте их на странице.
2. Реализуйте простой чат с использованием веб-сокетов (доп литературу для этого задания приложу позже).
3. Оптимизируйте страницу с большим количеством элементов, используя ленивую загрузку и `DocumentFragment`.

Если у вас остались вопросы или что-то не работает, напишите в telegram @ritouchka, я помогу разобраться! 😊